

STGraph: A **Multistore Data Structure** for Temporal Graph and Time Series

Matteo Francia
DISI – University of Bologna
Cesena, Italy
m.francia@unibo.it

Matteo Golfarelli
DISI – University of Bologna
Cesena, Italy
matteo.golfarelli@unibo.it

Manuele Pasini
DISI – University of Bologna
Cesena, Italy
manuele.pasini@unibo.it

ABSTRACT

With the ever-increasing amount of data produced by Internet of Things devices, organizations, and social networks, finding a proper way to store and query linked, evolving, and spatial time-series data is more and more complex. When it comes to their integration, an effective solution is not yet there. On the one hand, rigid relational implementations struggle with the heterogeneity of data to model, but provide efficient querying and data access. This is especially true for time series databases that allow for fast collection and storage of large volumes of data changing over time. On the other hand, graph implementations (such as knowledge or property graphs) are extremely flexible but struggle with the management of time series. In this paper, we introduce STGraph, a **multistore data structure solution** that enables to (i) store and query interconnected data along with their spatial and temporal dimensions and (ii) provide a versioning of the graph over time.

PVLDB Reference Format:

Matteo Francia, Matteo Golfarelli, and Manuele Pasini. STGraph: A **Multistore Data Structure** for Temporal Graph and Time Series. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

tramite il paper di T-GQL sono arrivato a questo paper <https://link.springer.com/article/10.1007/s00778-007-0058-x> che formalizza la stessa semantica temporale che usiamo noi di default, che viene definita "continuous path"

The widespread deployment of Internet of Things (IoT) infrastructures and the rapid adoption of digital twin technologies [1] have introduced new data management challenges that expose the limits of current graph- and time-series-based approaches [2], such as the integration of *evolving*, *linked*, (*spatial*) *time-series* data.

Example 1.1 (Healthcare). Bedside monitoring systems continuously produce extensive time series of vital signs and equipment usage over long periods of time [3]. Integrating time series with

graph databases allows longitudinal data to be linked with information such as patient histories, medical devices, and treatment protocols. This enables clinicians to analyze long-term trends and understand dependencies between patients and resources. □

Example 1.2 (Precision agriculture). Fleets of robots operating in agricultural fields (that belong to a farm, have different crop types, and are managed by different farmers) generate hundreds of georeferenced samples per minute [4]. By integrating time series with graph databases, these heterogeneous data streams can be efficiently correlated with master data of farms, fields, and crops to better understand temporal and spatial trends, transforming precision agriculture systems from mere monitoring tools into proactive, context-aware decision support frameworks. □

The examples emphasize the need for flexible hybrid representations. *Linked* data requires a flexible data model that enables efficient joins (e.g., to know which robot operates in a field). *Evolving* data demands an efficient versioning of historical data (e.g., to know which robots have operated in a field, and when). *Spatial time-series* data require engines that support spatial and temporal expressiveness (e.g., to identify the working areas along with the produced samples). On the one hand, Graph DBMSs excel at representing interconnected entities, enabling topological analysis. Yet, they lack optimized support for managing large-scale temporal data. On the other hand, time-series DBMSs efficiently manage large volumes of time-indexed data and excel in handling large volumes of dynamic data. However, they are ill-suited to capture the complex, time-evolving interactions between entities. Single model solutions (e.g., Neo4J or InfluxDB) are inefficient since they hardly reconcile the conflicting optimization goals (e.g., low-latency traversals versus high-throughput ingestion). Within such a complex landscape, new challenges are arising [5], and the research community is advocating for solutions exploring multimodel and multistore solutions [6].

Overview and contributions. In this paper, we introduce STGraph (Figure 1), a **multistore** for seamless and efficient management of temporal graph and spatial time series. In line with the multistore paradigm, we provide a uniform conceptual and querying perspective. From the user's point of view, all data is exposed through a single abstraction: a property graph, where *everything* is modeled through nodes, edges, and properties. The graph model natively supports the storage and querying of *schemaless* and *linked* data. *Evolving* data is modeled by versioning edges and nodes with temporal validity and by supporting temporal graph traversal. STGraph supports an efficient management of graph and *time series* (including the *spatial* dimension) data through a seamless integration of two different stores: a Graph Store and a Time-series Store.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

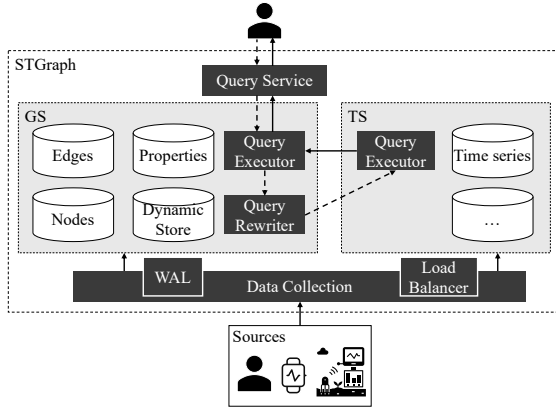


Figure 1: STGraph: straight arrows represent data flows, while dashed arrows represent query-rewriting flows.

At the bottom of Figure 1, data sources upsert data through the “Data Collection” service, which is responsible for dispatching the data to either GS or TS. Uploading data to TS does not require interaction with GS (and vice versa), enabling fully independent management of static/dynamic data. At the top of Figure 1, users query the (conceptual) graph through the “Query Service.” The “Query Service” interprets the incoming query and transforms it into an execution plan, which is forwarded to the “(GS) Query Executor,” also responsible for query optimization. During graph traversal, if the query requires data from TS, the “(GS) Query Executor” invokes the “Query Rewriter,” which translates and pushes the query down to the “(TS) Query Executor.” Finally, results are shuffled and aggregated by the “(GS) Query Executor,” which returns them to the “Query Service.”

The contributions of this paper are the following.

- (i) STGraph, a **hybrid data structure** integrating graph and spatio-temporal data (i.e., time series). The integration does not happen in middleware external to existing DBMSs, but is natively implemented by STGraph.
- (ii) Unified interface for querying spatio-temporal data using the property graph abstraction. The query is automatically translated into execution plans for graph and time series stores.
- (iii) Optimization of the query operators necessary to retrieve data from the time series store.
- (iv) Extensive testing with respect to state-of-the-art algorithms, showing that STGraph achieves up to **??** lower storage footprint and **??** lower execution time with respect to state-of-the-art competitors.

The paper is organized as follows. Section 2 positions and compares STGraph with the state-of-the-art contributions. Section 3 formally defines the spatio-temporal property graph. Section 4 introduces the distributed data structure. Section 5 describes the query execution and optimization. Section 6 extensively tests STGraph. Finally, Section 7 draws the conclusion and future research directions.

2 RELATED WORKS

@mpa aggiungere citazioni a lavori VLDB, survey TS, grafi temporali. Siamo un po' corti sul multistore e multimodel. secondo me siamo un po' corti su approcci TS DBMs. Non c'è una survey recente da citare? The management of schemaless, evolving, and time series data has received significant attention in the literature, and several solutions have been proposed to address these challenges. However, existing solutions tend to address only a subset of these aspects, falling short in effectively integrating and modeling the complexity of these data.

2.1 Single model solutions

2.1.1 Time series databases. Parallel with the rise of IoT and digital twins applications, time-series DBMSs have gained popularity in recent years [7]. Contrary to classic relational DBMSs, Time-series DBMSs usually rely on indexes, such as Log-Structured Merge Trees (LSM-Trees) [8], which buffer write operations in memory and flush them to disk in batches, drastically reducing the number of disk accesses and thus achieving exceptional ingestion performances. Compaction policies are then progressively performed to organize data as is flushed to disk.

InfluxDB [?] is among the most widely adopted Time Series DBMSs. InfluxDB’s primary focus lies in the efficient storage and retrieval of high-volume time-series, rather than in modeling the underlying entities. Several solutions have been proposed in the literature to address time-series data, such as Apache IoTB [9], AsterixDB [10] and Gorilla [11] x mf: meglio dire qualcosa di più ui due che cito ma non commento dopo limitandomi ad asterixdb?. Within these, AsterixDB is a shared-nothing distributed big data management system designed to efficiently manage large volumes of semi-structured data. It adopts a flexible data model based on nested, schema-less records and organizes information into datasets stored on LSM-tree-based storage structures. Each dataset is indexed and hash-partitioned across the nodes of the cluster using a primary key.

Overall, while time-series DBMSs excel in collecting and querying time-series data, offering native temporal (and even spatial) capabilities, they struggle to model the schemaless and complex relationships between the entities producing the data.

2.1.2 (Temporal) Graph databases. Graph DBMSs are well-suited for managing highly interconnected and dynamically evolving environments, such as IoT systems or social networks. Their data layout often relies on an index-free adjacency principle that enables efficient relationship navigation via direct neighbor access, making path traversal operations highly efficient. Neo4J [12], as the most popular graph DBMS, employs a storage layout built on an index-free adjacency data structure. However, it does not offer native support for temporal data and its performances tend to degrade significantly when handling large amounts of data, typical of modern data-intensive applications. Recent works have exploited LSM-trees for scalable graph data management [13, 14]. However, they are not temporal graphs since they lack versioning of graph elements.

Debrouvier et al. [15] provide, within T-GQL, a comprehensive classification of temporal graph solutions with respect to how temporal information is assigned to graph objects:

- *Duration-labeled* [16]: each **edge** is annotated with a duration of the relationship between two nodes. At a certain time, a node is considered valid if it participates in at least one edge that is valid at that time. This approach is mainly used for scheduling problems, where some shortest path has to be computed and they entail a fixed time granularity. **x mf: ovvero se la duration=3 implica che l'unità temporale è fissa, cosa che da noi non è necessaria.**
- *Interval-labeled* [15]: these temporal graphs could be seen as a generalization of duration-labeled graphs, in which each node/edge is annotated with a validity interval. This solution allows for a non-fixed time granularity and additional temporal query semantics (described in Section 3.2) at the cost of a small storage overhead. **x mf: da qualche parte dobbiamo parlare delle temporal query semantics e di quale scegliamo noi, non so se il punto sia qui in related, perchè sono sempre riassunte nel paper di T-GQL, o allargando sezione 3.2 con una forward reference. Inoltre, anche in T-GQL parlano di versionare solo gli archi e le property dei nodi in un interval-labeled graph, ma non i nodi in realtà, non ricordo again la spiegazione che mi avevi dato sul perchè versioniamo anche i nodi. Alla fine è un arco che inevitabilmente mi porta solo nei nodi validi a quell'intervallo di tempo.**
- *Snapshot-based* [17–20]: represent the graph as a sequence of discrete snapshots at different interval times. **x mf: la questione di come gestire lo storage è ortogonale a questa classificazione che è relativa a come modellare il tempo, e mentre x snapshot based graph è facile farla (vedi sotto), T-GQL al contrario poggia su Neo4J, e infatti non si identifica in nessuno degli storage perchè il tempo lo modellano come property degli elementi e non a livello fisico... Non so se abbia davvero senso affrontarla, sarebbe carino ma non ho trovato nulla atm. Credo si possa parlare di tuple-timestamping, in quanto non teniamo traccia degli update ma ogni elemento ha associato a livello fisico la sua validità.. Ha senso parlarne? Nel caso è un altro distinguo aggiuntivo rispetto ad approcci esistenti. Several physical storage strategies have been proposed for snapshot-based temporal graphs, including Copy+Log [18], δ -Copy+Log [19], and Anchor+Delta [20]. Despite each approach optimizes its predecessor by improving space efficiency through redundancy reduction, snapshot-based representations entail significant storage overhead, which limits their scalability when managing highly dynamic data and large-scale graphs.**

With respect T-GQL, which models temporal information through standard property graph properties; STGraph introduces a physical representation of element-level timestamping **x mf: non mi piace molto element-level timestamping** where the temporal validity is physically stored as a first citizen of a graph item and its properties. With respect to Clock-G [19] and AeonG [20], ST-Graph follows an interval-labeled semantics and limits its storage overhead through tuple-based physical temporal data. With respect to performances and to the best of our knowledge, AeonG [15] is the state of the art in terms of temporal graphs. Its parquet-based

anchor+delta design balances storage efficiency and query performance by reducing redundant data and limiting read amplification during temporal queries. Finally, the mentioned approaches, including AeonG, lack native spatial expressiveness and time series management.

Additionally, the evaluation of AeonG is restricted to relatively small datasets, focusing primarily on small-range and point temporal queries.

2.2 Multimodel and multistore solutions

2.2.1 Multimodel. Graphix [21] extends AsterixDB by providing a graph, where nodes are views over datasets and edges are views that link/join datasets. Nonetheless, Graphix does not natively support temporal graphs and builds on a logical view: its storage remains based on AsterixDB's document-oriented model, and graph structures are computed at query time. **Specifically, each edge traversal entails a join operation between two existing dataset, making it unsuitable for large graph traversal operations. quello che mi è venuto in mente per giustificare l'esclusione è che noi ci confrontiamo con AGTS che ha gli archi materializzati in tabelle, qui per ogni traversal dovresti fare un join... Teoricamente by default sei più lento** z Hygraph [22] is a hybrid in-memory data structure that integrates temporal graph and time series (modeled as graph properties), and introduces operators to create graph overlays from time-series data. Being in-memory, Hygraph is efficient in graph ingestion but is very limited by main-memory capacity.

2.2.2 Multistore. AGTS is multistore solution based on PostgreSQL [23] extended with Apache AGE [?], Timescale [?], and PostGIS [?]. PostGIS enhances PostgreSQL with spatial capabilities, TimescaleDB introduces efficient time-series data management via hypertables, and Apache AGE introduces graph capabilities through a hybrid query language (SQL functions that parse Cypher syntax). Graph data is implemented as logical abstractions on top of PostgreSQL's relational storage model, where each node and edge label is mapped to a corresponding relational table (graph traversals are executed through relational operators rather than native graph-processing mechanisms). **To the best of our knowledge, no research paper has provided a full-fledged multistore solution as STGraph.**

3 SPATIO-TEMPORAL PROPERTY GRAPH

A spatio-temporal **noi siamo un interval-labelled temporal property graph** property graph extends the traditional property graph, incorporating both temporal and spatial dimensions.

Definition 3.1 (Property graph). A property graph is a tuple: $G = (V, E, L, P, \ell, p)$ where: V is the set of nodes, $R \subseteq V \times V$ is the set of directed edges, L is the set of labels, P is the set of key-value properties, $\ell : V \cup E \rightarrow L$ maps each node/edge to a label, $p : V \cup E \rightarrow P$ maps each node/edge to a set of key-value properties.

Given the temporal T (i.e., a finite set of consecutive timestamps) and spatial S domains, we extend Definition 3.1 to a spatio-temporal property graph.

Definition 3.2 (Spatio-temporal property graph). A spatio-temporal property graph $G^{st} = (V, E, L, P, \ell, p, \tau, \phi)$ is an extension of a property graph $G = (V, E, L, P, \ell, p)$, where

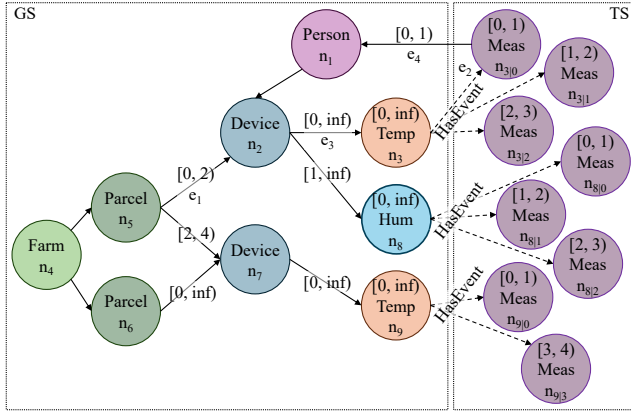


Figure 2: Example of the user-perceived graph for the precision agriculture domain. Different colors represent different node labels, and temporal validity is reported for nodes/edges (where omitted, the validity is $(-\text{inf}, \text{inf})$). Node n_1 (“Alice”, with label Person) is the owner of n_2 (“Device 1”, Device) (e.g., a weather station) that produces two time series n_3 (“Temperature 1”, Temperature) and n_8 (“Humidity 1”, Humidity). n_4 (“Farm 1”, Farm) is composed of n_5 (“Parcel 1”, Parcel) and n_6 (“Parcel 2”, Parcel) that share n_7 (“Device 2”, Device) producing a time series n_9 (“Temperature 2”, Temperature). n_5 also contains n_2 . Virtual edges with label HasEvent are not physically stored but dynamically generated.

- $\tau : V \cup E \cup P \rightarrow \mathbb{T}^2$ assigns temporal interval validity $[from, to]$ to nodes, edges, and properties, enabling interval-labeled temporal graph.
- $\phi : V \rightarrow \{0, 1\}$ returns whether a node is a time series.
- $(\text{location}, v), v \subset \mathbb{S}$ is a property that assigns spatial information to nodes and edges.

Note that location is a property and not a first citizen of a node/edge since nodes/edges can change locations over time (e.g., a person changing their home address over time).

Example 3.3. Let G^{st} represent a spatio-temporal property graph modeling Figure 2. The set of nodes V contains n_2 (a weather station), n_5 (a parcel), and $n_{3|0}$ (a measurement). The set of edges E contains $e_1 = (n_5, n_2)$ and $e_2 = (n_3, n_{3|0})$. Examples of labels, properties, and temporal validity are:

$$\begin{aligned} l(n_5) &= \text{Device} \\ l(e_1) &= \text{InstalledIn} \\ l(e_2) &= \text{HasEvent} \end{aligned}$$

$$\begin{aligned} p(n_2) &= \{(\text{name}, \text{“Device 1”}), (\text{type}, \text{“Weather Station”})\} \\ p(n_5) &= \{(\text{name}, \text{“Parcel 1”}), (\text{location}, \text{Polygon}(\dots))\} \\ p(n_{3|0}) &= \{(\text{value}, 16), (\text{unit}, \text{“C”}), (\text{location}, \text{Point}(\dots))\} \\ \tau(n_{3|0}) &= [0, 1] \\ \tau(e_1) &= [0, 2] \end{aligned}$$

□

3.1 Operators

We distinguish between the operators necessary to build and query the spatio-temporal property graph.

3.1.1 Building the graph. Building the spatio-temporal property graph is based on the following operators.

- (1) **create(v)** (creating a node) adds v to the graph (i.e., $G.V = G.V \cup \{v\}$) with a time period $\tau(v) = [from, to]$. Unless otherwise specified, $to = \text{inf}$.
- (2) **create(e)** (creating an edge) adds $e = (v_1, v_2)$ to the graph (i.e., $G.E = G.E \cup \{e\}$) with a time period $\tau(e) = [from_e, to_e]$, where $\tau(e)$ is bounded by the time spans $\tau(v_1) = [from_{v_1}, to_{v_1}]$ and $\tau(v_2) = [from_{v_2}, to_{v_2}]$. Formally, $from_e \geq \max(from_{v_1}, from_{v_2})$ and $to_e \leq \min(to_{v_1}, to_{v_2})$. Unless otherwise specified, $to_e = \text{inf}$ (i.e., it has no end validity).
- (3) **upsert(x, p)** (update/add a to node/edge x with a property p) adds p to the graph (i.e., $G.P = G.P \cup \{p\}$) with a time period $\tau(p) = [from_p, to_p]$, where $\tau(p)$ is bounded by $\tau(x) = [from_x, to_x]$. Formally, $from_p \geq from_x$ and $to_p \leq to_x$. Note that (i) properties are immutable, and (ii) adding a property does not update the validity of the node/edge, since its validity could be longer than that of the single property (but the property cannot exceed the validity of the node/edge to which it belongs).
- (4) **delete(x)** (deleting a node/edge/property) changes its temporal validity from $\tau(x) = [from, to]$ to $\tau(x) = [from, to']$, where $to' < to$. When a node is deleted, all its edges and properties are deleted as well. When an edge is deleted, all its properties are deleted as well.

To be valid, a spatio-temporal property graph requires temporal constraints to be satisfied, but it does not require any spatial constraint (e.g., a weather station installed in a farm can “cover” a surface larger than the farm).

3.1.2 Querying the graph. As in [24], we consider *linear* graph queries, i.e. queries where each clause is an operator that takes a relation and outputs a relation that can both expand the number of attributes and add new tuples. The linear order is declarative, and execution plans can re-order the execution of the operators without affecting the result. Given a spatio-temporal property graph G , a relation R is a set of tuples $\{(\dots, x, \dots), \dots\}$ where $x \in G.V \cup G.E \cup G.P$ (elements in the same position are either nodes, edges, or properties), the header of a relation $\mathcal{A}(R) = (\dots, id, \dots)$ is a tuple of arbitrary ids.

Example 3.4. With reference to Figure 2, a relation R with header $\mathcal{A}(R) = (\text{Parcel}, \text{Device}, \text{Value})$ is

Parcel	Device	Value
n_5	n_2	(value, 12)
n_5	n_2	(value, 20)
n_6	n_7	(value, 15)

□

An execution plan for a graph query can be modeled as the composition of relational-algebra operators and an additional expand operator [24]. We list the operators that are necessary in the remainder of the paper; the full formalization can be found in [25].

- (1) selection filters graph elements using a predicate θ : $\sigma_\theta(R) = \{t \in R \mid \theta(t)\}$.
- (2) projection restricts a relation to a subset of its attributes: $\pi_A(R) = \{t[A] \mid t \in R\}$, where $A \subseteq \mathcal{A}(R)$, and can replace a node/edge with one of its properties.
- (3) join combines two relations using a predicate θ : $R_1 \bowtie_\theta R_2 = \{t_1 \cup t_2 \mid t_1 \in R_1, t_2 \in R_2, \theta(t_1, t_2)\}$
- (4) expand traverses edges starting from a set of vertices. With respect to original relational algebra, expand is a graph-specific operator that explicitly models edge traversal in two directions. The *outgoing* expand is $\text{expand}_\theta^{\rightarrow}(V) = \{(v, e, v') \mid v \in V, v' \in V, e = (v, v') \in E, \theta(e, v')\}$; dually, the *incoming* expand is $\text{expand}_\theta^{\leftarrow}(V) = \{(v, e, v') \mid v \in V, v' \in V, e = (v', v) \in E, \theta(e, v')\}$. expand is similar to a relational join: it finds pairs of nodes that are connected through an edge. However, it uses pointers to reach the next node without reading any unnecessary data or proceeding via an indirection such as an index. For the sake of brevity, henceforth we refer to expand without making explicit reference to its direction.
- (5) aggregation groups tuples of a relation and applies an aggregation function: $\gamma_{A, f(a)}(R)$ where: $A \subseteq \mathcal{A}(R)$ is a set of grouping attributes, $f \in \{\text{COUNT, SUM, MIN, MAX, AVG}\}$, and $a \in \mathcal{A}(R)$.

3.2 Temporal semantics

Qui secondo me si potrebbe parlare, sempre citando T-GQL delle semantiche di temporal path query introdotte nella presentazione con AB: continuous path semantics (quella adottata da noi), consecutive path semantics e pair-wise path semantics. Allo stato attuale supportiamo solo la continuous, ma potenzialmente le supportiamo tutte. Solving linear graph queries in spatio-temporal property graphs also requires validating temporal constraints. The overall validity of a tuple $t = (\dots, x, \dots)$ in a relation R is $\tau(t) = \bigcap_{x \in t} \tau(x)$, $r \in T$ and the relation R only contains tuples such that $\tau(t) \neq \emptyset$. Note that, since the intersection operator is commutative and associative, this constraint is compliant with the declarative nature of linear graph queries having execution plans that can re-order the execution of clauses without affecting the final result.

Extending a relation. When extending a relation through the join and expand operators, the new relation contains only tuples having elements with intersecting validity.

Example 3.5. In Figure 2, nodes and edges have different temporal validity. For instance, n_2 was attached to n_5 in $[0, 2)$, but then was replaced with n_7 in $[2, 4)$. Given the graph query (henceforth, queries will be expressed in the Cypher language for the sake of simplicity and readability)

```
1 MATCH (p: Parcel)-->(d: Device)-->(t)-->(m: Measurement)
2 WHERE p.name = "Parcel 1"
3 RETURN p, d, t, m.value
```

the result is

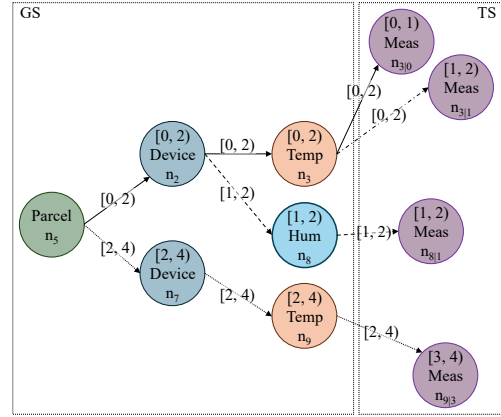


Figure 3: Patterns in the query result with monotonically-decreasing temporal validity (from left to right).

p	d	t	value
n_5	n_2	n_3	(value, 16)
n_5	n_2	n_3	(value, 19)
n_5	n_2	n_8	(value, 86)
n_5	n_7	n_9	(value, 17)

Figure 3 shows the temporal validity (evolving from left to right) of the tuples composing the final relation. $\square$

Projecting a relation. When $\text{project}_\theta(R)$ produces a new relation R' by transforming a node/edge in R into a property, it could also be necessary to dynamically generate new tuples in R' due to the temporal validity of the transformed property. Given the tuple $t = (\dots, x, y, \dots)$ where $x, y \in G.V \cup G.E$, the dynamic generation works by iteratively applying the *temporal property join* algorithm described in Algorithm 1. Algorithm 1 computes a temporal join between properties of two graph elements x and y by aligning their validity intervals. First, the properties of x and y matching the given name are extracted from the graph (Lines 1-2). All temporal boundaries induced by these properties are then collected and sorted to define a sequence of consecutive time intervals (Line 3). The result accumulator is initialized (Line 4), and the earliest boundary is selected and removed (Line 5). The algorithm then iterates over the next boundaries (Line 6), each time defining a temporal slice $[cur, next)$ (Line 7). For each slice, it identifies the properties of x and y whose temporal intervals overlap the current slice (Lines 8-9). The Cartesian product of these temporally valid properties is computed and accumulated into the result set (Line 10). Once all temporal slices have been processed, the aggregated set of joined property pairs is returned (Line 13).

Example 3.6. With reference to Figure 2, let us assume that n_5 had the property name changed over time

$$\begin{aligned}
 p(f_1) &= \{p', p''\} \\
 p' &= (\text{name, "Farm 1"}, \quad \tau(p') = [0, 2) \\
 p'' &= (\text{name, "Farm 01"}, \quad \tau(p'') = [2, \text{inf})
 \end{aligned}$$

The query `MATCH (f: Farm)-->(p) RETURN f, p` returns

Algorithm 1 Temporal property join

Require: *name*: name of the property to replace, $x, y \in G.V \cup G.E$
Ensure: *agg*: pairs of properties

```

1:  $P_x \leftarrow \{(k, v), k = \text{name}, (k, v) \in p(x)\}$   $\triangleright$  Get/filter the properties
2:  $P_y \leftarrow \{(k, v), k = \text{name}, (k, v) \in p(y)\}$   $\triangleright$  Get and sort the boundaries of all temporal intervals
3:  $\text{boundaries} \leftarrow \text{sort}(\bigcup_{p \in P_x \cup P_y} \{\tau(p).from, \tau(p).to\})$ 
4:  $\text{agg} \leftarrow \emptyset$ 
5:  $\text{cur} \leftarrow \text{pop}(\text{boundaries})$   $\triangleright$  Lower bound of the time interval
6: while  $\text{boundaries} \neq \emptyset$  do
7:    $\text{next} \leftarrow \text{pop}(\text{boundaries})$   $\triangleright$  ... and upper bound
    $\triangleright$  Find valid properties; possibly none, if so  $\hat{P} \leftarrow \{\emptyset\}$ 
8:    $\hat{P}_x \leftarrow \{p, \tau(p) \cap [\text{cur}, \text{next}) \neq \emptyset, p \in P_x\}$ 
9:    $\hat{P}_y \leftarrow \{p, \tau(p) \cap [\text{cur}, \text{next}) \neq \emptyset, p \in P_y\}$ 
    $\triangleright$  Compute and accumulate their Cartesian product
10:   $\text{slices} \leftarrow \text{slices} \cup (\hat{P}_x \times \hat{P}_y)$ 
11:   $\text{cur} \leftarrow \text{next}$ 
12: end while
13: return agg

```

<i>f</i>	<i>p</i>
n_5	n_2
n_5	n_7

When project is applied, **MATCH** ($f: \text{Farm}$) $\rightarrow$ (p) **RETURN** $f.\text{name}$, $p.\text{name}$ returns

<i>f.name</i>	<i>p.name</i>
$\emptyset$	(name, "Parcel 1")
(name, "Farm 1")	(name, "Parcel 1")
(name, "Farm 1")	(name, "Parcel 2")
$\emptyset$	(name, "Parcel 2")
(name, "Farm 01")	(name, "Parcel 1")
(name, "Farm 01")	(name, "Parcel 2")

The first tuple $t_1 = (\emptyset, (\text{name}, \text{"Parcel 1"}))$ has $\tau(t_1) = (-\text{inf}, 0)$, $t_2 = ((\text{name}, \text{"Farm 1"}), (\text{name}, \text{"Parcel 1"}))$ has $\tau(t_2) = [0, 2)$ and, $t_3 = ((\text{name}, \text{"Farm 1"}), (\text{name}, \text{"Parcel 1"}))$ has $\tau(t_3) = [2, \text{inf})$. $\square$

4 STGRAPH

Following a multistore approach, our goal is to provide uniform conceptual data abstraction. We rely on the conceptual model of property graph since it is suitable to navigate heterogeneous data with temporal and spatial characteristics. From the user perspective, *everything is a graph composed of nodes, edges, and properties*.

Under the hood, we leverage a hybrid data structure that is composed of (i) a centralized index-free adjacency Graph Store (GS) to memorize and query interconnected data, and (ii) a distributed Time-series Store (TS) to memorize and query spatio-temporal data. More specifically, TS is designed to handle large volumes of temporal data by supporting fast reads and concurrent writes. Indeed, the amount of time series data is orders of magnitude bigger than graph data (e.g., the amount of data produced by an autonomous car vs the data describing the car).

4.1 Graph Store

The Graph Store (GS) stores nodes, edges, and properties as fixed-size records in independent in-memory data structures whose ACID properties and persistence are guaranteed through a write-ahead log. The structures of GS *nodes*, *edges*, and *properties* are summarized

Table 1: Data structures of GS and TS. Size is in Bytes.

GS Nodes		GS Edges	
Field	Size	Field	Size
Id	8	Id	8
From (timestamp)	8	From (timestamp)	8
To (timestamp)	8	To (timestamp)	8
Label	4	Label	4
Next edge	8	Next edge (from node)	8
Next property	8	Next edge (to node)	8
Is ts (i.e., ϕ)	1	Next property	8
Total	45	From node	8
		To node	8
		Total	68

GS Properties		Time series	
Field	Size	Field	Size
Id	8	Id (from timestamp)	8
From (timestamp)	8	Label	4
To (timestamp)	8	Value	8
Next property	8	Location	Any
Source id	8	Edges	Any
Source type	1	Properties	Any
Type	4	Total	Any
Key	24		
Value	8		
Total	77		

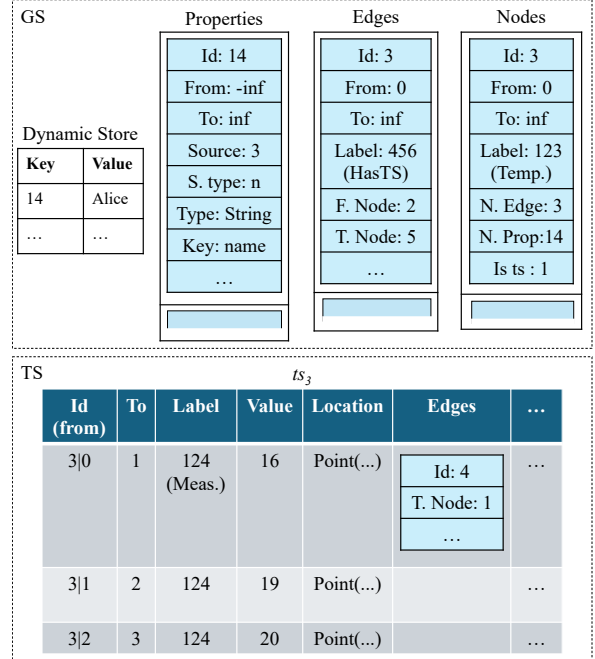


Figure 4: Excerpt of data structure for Figure 3.

in Table 1 and depicted in Figure 4. If the property value fits 8 bytes (e.g., integer, long, or double values in Java or a short UTF-8 String), the value is directly stored in the property. Otherwise, the value is stored in a key-value Dynamic Store (implemented in RocksDB) where the key is the "property id."

Pointers to nodes and fixed-size records allow for index-free adjacency join resolution, i.e. join queries are solved by traversing edges, making this storage model optimized for high-performance

traversal without scanning unrelated data or performing costly joins.

4.2 Time-series Store

Time series are long series of events, mostly append-only, and ordered by time. TS memorizes time series along with unique identifiers, and each event of a time series is a variable-size “fat” event/node with nested edges/properties (rather than storing pointers to edges/properties). Events/nodes of the same time series are stored in the same collection (resulting in one collection for each time series). The structure of TS *nodes* is summarized in Table 1 and depicted in Figure 4. Collections are indexed by time (and space as a secondary index) using LSM-tree indexes (Log-Structured Merge) [8]. LSM-tree indexes handle highly intensive ingestion workloads efficiently by buffering write operations in memory and periodically flushing them onto disk in batches. This makes LSM-tree indexes beneficial for continuously ingesting large volumes of data.

4.3 Connecting the stores

Connecting GS and TS requires (i) transparent data access and (ii) seamless navigation of data stored in two different layouts. Note that node IDs are built such that the ID is sufficient to determine its source (GS or TS stores) and its offset within the source. Given a node v and its temporal validity $\tau(v) = [from, to]$, $v.id$ is a memory offset in the case of the graph, and $v.id = from$ in the case of TS.

4.3.1 Data collection. Through the “Data Collection” service (Figure 1), the management of each time series is independent of the graph and other time series (given the time series id, data sources can directly upload data into the correct time series. Given a GS node v (e.g., a sensor recording 1 measurement per second) such that $\phi(v) = true$, when a new event e_1 (i.e., a TS node) is stored (e.g., at the first recorded measurement), it is necessary to (i) create a new time series $ts \in TS$ where $ts.id = v.id$, and (ii) append e_1 to ts . When a new event e_i of v is recorded (e.g., after the first recorded measurement), v does not have to be updated since it already refers to ts , and e_i can be independently appended to ts .

4.3.2 Querying. The seamless querying of GS and TS is carried out by the “Query Service” (Figure 1).

Traversing from GS to TS. Since the number of nodes in TS (e.g., sensor measurements) is sensibly higher than the number of nodes in GS (e.g., master data describing the sensors), storing the edges going from GS to TS (HasEvent in Figure 2) would require materializing an extremely large number of edges. Such a design would not only incur a prohibitive storage overhead but would also negatively impact traversal efficiency. Instead, we introduce the notion of *virtual edges*, which are not explicitly stored in the graph structure but are generated at query time (see the dashed edges with label HasEvent in Figure 1).

Virtual edges are dynamically generated when traversing a GS node v with the expand operator and $\phi(v) = true$. The query jumps from GS to the correct time series $ts \in TS$, $ts.id = v.id$. This drastically reduces storage requirements while preserving the connectivity between GS and TS.

To ensure the correctness of virtual edge resolution without incurring the cost of materializing a massive number of edges, we impose the following constraints:

- Each GS node refers to a single time series. When multiple time series should be associated with the same node v , they can be represented as multiple nodes, each referring to exactly one time series, and linked to v .
- GS to TS navigation is currently restricted to the node to which the time series belongs (this is necessary to avoid edge duplication in GS and TS).
- Virtual edges (HasEvent in Figure 2) do not have properties, as they are not physically stored in the graph.

Traversing from TS to GS. Time series in TS contain (i) an *id* corresponding to the GS node v to which the time series belongs (i.e. $v.id = ts.id$), and (ii) “fat” events (i.e., TS nodes with nested edges/properties). Given a TS node v , a query will traverse from TS to GS by navigating valid (filtered) $v.edges$. For the sake of performance, TS-to-TS traversal is currently prevented since it requires joining (long) time series.

4.4 Versioning in STGraph

The way nodes, edges, and properties are stored in STGraph, along with their temporal validity (as in interval-labeled temporal graphs, see Section 2) prevents the need to persist redundant versions of the elements of the spatio-temporal property graph. This eliminates the need to manage snapshots and delta updates (see Section 2), which would otherwise require persisting extra information. For instance, in Figure 2, n_2 was added to n_5 and then removed, resulting in an edge e with temporal validity of $\tau(e) = [0, 2]$; this versioning does not require adding any additional edge/node to the graph. The way in which versioning affects query operators is described in Section 3.2.

5 QUERY EXECUTION AND OPTIMIZATION

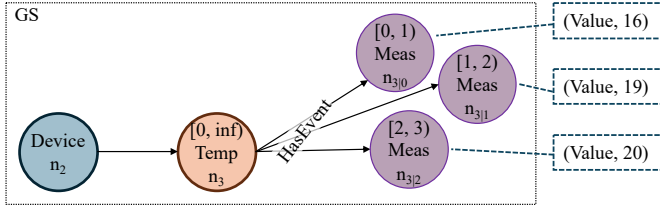
Users query the spatio-temporal property graph through the “Query Service” (Figure 1), which is responsible for creating an execution plan. In this paper, we do not provide algorithms for global query plan optimization, but provide optimizations of the single query operators. Given a query (written in the Cypher language, for the sake of readability and simplicity), the “Query Service” solves the **MATCH** clause from left-to-right, it aggregates filters on the same node/edge (expressed in both **MATCH** and **WHERE** clauses, and finally aggregates the result as described in **RETURN**.

Example 5.1. For the following query

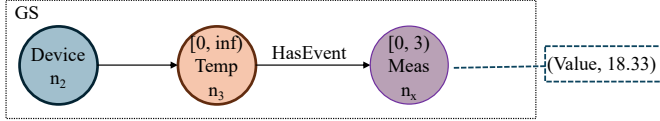
```
1 MATCH (d:D {"name": "Device 1"})-->(t)-->(m: M)
2 WHERE m.value > 19
3 RETURN d, avg(m.value)
```

The “Query Service” (i) filters all nodes having label D and property (name, “Device 1”), (ii) finds all the connected nodes, (iii) finds all connected nodes having label M and value > 19, and (iv) aggregates by device all the measurement values. □

The “Query Service” creates an execution plan for the query and forwards it to the “(GS) Query Executor.” For queries spanning both GS and TS, a simple “(GS) Query Executor” would aggregate all necessary events from TS into main memory and process them



(a) No optimization: all plain events are collected back to GS and averaged in post-processing.



(b) Optimization: events are averaged in TS, and the result is collected back to GS.

Figure 5: Example of optimization with reference to Figure 2: aggregating events. When “(GS) Query Executor” traverses n_3 , in (a) no optimization is applied, and (b) aggregation is pushed down to TS.

(Figure 5a). However, since TS contains orders-of-magnitude more data than GS and is distributed across different machines, this would require shuffling all these events back to GS and implies that they fit the main memory.

In our implementation, when the “(GS) Query Executor” realizes that an operator requires data from TS, the execution of the operator is rewritten by the “Query Rewriter” and demanded to the “(TS) Query Executor.” Only the result of the execution (and *not* all the plain events; Figure 5b) is collected back by the “(GS) Query Executor” and converted to nodes.

We now discuss the optimization of each operator and provide an example of a query that benefits from the optimization with reference to Figure 2.

Selection $\sigma_\theta(R)$. All predicates in θ that involve only the elements of a time series in TS can be pushed down to TS.

Example 5.2. Samples above 19 degrees are filtered in TS.

```
1 MATCH (t: Temperature)-->(m: Measurement)
2 WHERE m.value > 19
3 RETURN t, m
```

Time series having at least one measurement above 19 degrees are filtered in TS with an existence condition.

```
1 MATCH (t: Temperature)-->(m: Measurement)
2 WHERE m.value > 19
3 RETURN t
```

□

Projection $\pi_A(R)$. Only the elements A required to solve a graph query are selected in TS and shuffled to GS.

Example 5.3. Only the property value is shuffled back to GS.

```
1 MATCH (d: Device)-->(t: Temperature)-->(m: Measurement)
2 RETURN d, t, m.value
```

□

Join $R_1 \bowtie_\theta R_2$. Join is differently optimized depending on θ .

Index join. If θ involves indexed attributes in TS (e.g., node’s id or location), an index join is applied on the primary key index (for id) or the secondary index (for location). The overall complexity is $O(|R_1| \cdot \log(|R_2|))$.

Example 5.4. Getting samples with the same timestamp requires joining on timestamp/id, which is indexed as the primary key of the time series.

```
1 MATCH (d: Device)-->(t: Temperature)-->(m1: Measurement),
2 (d)-->(h: Humidity)-->(m2: Measurement)
3 WHERE m1.id = m2.id
4 RETURN m1.id, m1.value, m2.value
```

□

Example 5.5. Getting the samples of temperatures collected in a parcel requires a spatial join based on the location property, which is a secondary index of a time series.

```
1 MATCH (p: Parcel), (h: Temperature)-->(m: Measurement)
2 WHERE ST_CONTAINS(p.location, m.location)
3 RETURN m.value
```

□

Nested loop. Otherwise, a nested loop can be applied, with an overall complexity of $O(|R_1| \cdot |R_2|)$.

Example 5.6. Getting only the samples of temperatures that are above a parcel-specific threshold.

```
1 MATCH (p: Parcel), (h: Temperature)-->(m: Measurement)
2 WHERE p.threshold > m.value
3 RETURN m.value
```

□

Traversal $\text{expand}_\theta(V)$. Traversal is differently optimized depending on the source node v .

From GS to TS. If v is a GS node and $\phi(v) = \text{true}$ (i.e., v has a corresponding time series $ts \in TS$), then triples (v, e, v') are dynamically generated by querying the time series $ts \in TS$ where $ts.id = v.id$ and by pushing down θ to ts .

Example 5.7. The edges connecting a time series with its events are not stored in GS but dynamically generated at query time. Events are directly queried from the time series in TS.

```
1 MATCH (t: Temperature)-[e]->(m: Measurement)
2 RETURN t, e, m
```

□

From TS to GS. If v is a TS node, then triples (v, e, v') are generated for each edge e in $v.edges$ satisfying θ .

Example 5.8. Measurements can be linked to nodes in GS.

```
1 MATCH (t: Temperature)-->(m: Measurement)-->(p: Person)
2 RETURN m, p
```

□

Aggregation $\gamma_{A, f(A)}(R)$. Aggregation is pushed down to TS if events have no outgoing graph dependency (i.e., they are the right-most element of the **MATCH** clause).

Example 5.9. With reference to Figure 5b, the average temperature can be pushed to TS, with $\gamma_{G, f(A)}(R) = \gamma_{\{t\}, \text{AVG}(m.value)}(R)$

Table 2: Characteristics of SOTA approaches.

	Expressiveness		Engine		Deployment	
	Spatial	Temporal	Graph	Time-series	Distributed	Persistent
Neo4J			✓			✓
AGTS	✓			✓		✓
Hygraph		✓		✓		
AeonG		✓	✓			✓
STGraph	✓	✓	✓	✓	✓	✓

Table 3: Benchmark statistics.

Benchmark	Size	GS		TS		Upserts
		Nodes	Edges	Events	Avg. events / TS	
SmartBench	S	$3.4 \cdot 10^3$	$6.7 \cdot 10^3$	$2.5 \cdot 10^6$	$2.6 \cdot 10^4$	$2.5 \cdot 10^6$
	M	$6.0 \cdot 10^3$	$1.2 \cdot 10^4$	$1.0 \cdot 10^7$	$5.2 \cdot 10^4$	$1.0 \cdot 10^7$
	L	$2.6 \cdot 10^4$	$5.5 \cdot 10^4$	$2.6 \cdot 10^8$	$2.6 \cdot 10^5$	$2.6 \cdot 10^8$
	XL	$2.6 \cdot 10^4$	$5.5 \cdot 10^4$	$6.2 \cdot 10^8$	$1.5 \cdot 10^6$	$1.5 \cdot 10^9$
SocialNetwork	S	TODO	TODO	TODO	TODO	TODO

Table 4: Characteristics of query workload.

Benchmark	ID	Store		Operators				
		GS	TS	Join	Aggregate	Select	Expand	Project
SmartBench	Q1	✓				✓	✓	✓
	Q2	✓	✓	✓	✓	✓	✓	✓
	Q3	✓	✓	✓		✓	✓	✓
	Q4	✓	✓	(spatial)	✓	✓	✓	✓
	Q5	✓	✓		✓	✓	✓	✓
	Q6	✓			✓	✓	✓	✓

```

1 MATCH (t: Temperature)-->(m: Measurement)
2 RETURN t, AVG(m.value)

```

whereas

```

1 MATCH (t: Temperature)-->(m: Measurement)-->(p: Person)
2 RETURN t, AVG(m.value)

```

cannot be directly pushed to TS since, before grouping events, it is necessary to filter the ones connected to a person. □

?? Spiegare come si creano i nodi virtuali aggregati, anche nel rispetto delle logiche temporali ??

?? noi non valutiamo gli improvement delle nostre ottimizzazioni, ma solo il confronto con gli altri ??

6 EXPERIMENTAL EVALUATION

In STGraph, GS has been developed in Kotlin and TS has been implemented in AsterixDB [10]. Each time series is physically stored as an AsterixDB dataset of records partitioned across nodes in a cluster and indexed through LSM-tree indexes [26]. AsterixDB also supports secondary indexes, including a spatial LSM R-tree [26] used to index the location property. The replication factor in AsterixDB was fixed to one to take into account raw storage costs only.

STGraph leverages AsterixDB Data Feeds [27] to implement high-throughput ingestion adapters for time-series data. A Data Feed allows the definition of an efficient ingestion pipeline that maps incoming data from an external source (operating in either

push or pull mode) directly into existing datasets. STGraph assigns a dedicated data feed to each time-series node, enabling parallel streaming ingestion across multiple data sources.

The implementation of STGraph is available at <https://github.com/w4bo/dt-graph> and the full testing pipelines are available for reproducibility on GitHub (<https://github.com/big-unibo/stgraph-evaluation>). Experiments were conducted on a cluster of machines equipped with an Intel Xeon W5-2445 processor (20 cores, 4.6 GHz) and 256 GB of DDR5 RAM.

STGraph is compared against state-of-the-art approaches on temporal graphs and time series, namely AeonG [15], Hygraph [22] (in-memory only), Neo4J (single-model solution based on property graph), and AGTS (multimodel solution extending postgres with Apache Age and TimescaleDB). AeonG and Hygraph are native solutions for temporal graphs. In Neo4J, time series events are treated as graph nodes. In AGTS, graph data is handled through Apache Age and time series events through TimescaleDB.

6.1 Datasets

For the testing datasets, Table 3 summarizes the number of nodes and edges in GS, events in TS (including the average time-series length), and insert/update operations necessary to create the spatio-temporal property graph.

6.1.1 SmartBench. SmartBench [28] is a benchmark derived from a real-world smart building monitoring system, and was specifically designed to investigate the data management requirements of pervasive sensing environments. SmartBench models devices composed of sensors performing environmental measurements, the spatial structure of the monitored environment (e.g., buildings and rooms), and the high-frequency measurements produced by the deployed sensors. Sensors and environments map to a temporal graph based model, while sensor measurements exhibit a time-series nature. SmartBench provides a data generation tool to construct datasets of increasing scaling factors: Small ($\times 1$), Medium ($\times 2$), and Large ($\times 10$); XL is a variation of L were we extended the length of time series.

SmartBench also defines a set of queries that differ in terms of filtering conditions, aggregation functions, and selection criteria. The characteristics of the queries selected for evaluation are the detailed in Table 4 (the complete specification of the queries can be found in ???). Q1 and Q6 are graph traversal queries (i.e., they do not require access to TS); Q1 works on the whole graph spanning the entire temporal domain, while Q6 applies filters to the temporal domain to get specific historical relationships. Q2 and Q5 are analytical queries; Q2 has a high-selectivity range filter (it includes 12% of time series events), while Q5 has a low-selectivity filter (it includes 37% of time series events). Q3 joins graph nodes and time series events based on existence conditions on specific events. Q4 involves a spatial join operation, since a device may produce data within an environment without being explicitly connected to it through a graph edge.

6.1.2 SocialNetwork. ?? @mpa TODO, deve essere simmetrico rispetto a SmartBench ??

Table 5: Ingestion time (s).

Benchmark	Size	AGTS	AeonG	HyGraph	Neo4J	STGraph
SmartBench	S	$1.6 \cdot 10^2$	$1.1 \cdot 10^8$	$3.5 \cdot 10^1$	$4.3 \cdot 10^4$	$1.1 \cdot 10^2$
	M	$9.6 \cdot 10^2$	$1.1 \cdot 10^9$	$8.5 \cdot 10^1$	–	$3.4 \cdot 10^2$
	L	$1.2 \cdot 10^5$	–	–	–	$4.4 \cdot 10^3$
SocialNetwork	S	TODO	TODO	TODO	TODO	TODO
	M	TODO	TODO	TODO	TODO	TODO
	L	TODO	TODO	TODO	TODO	TODO

Table 6: Storage footprint (MB).

Benchmark	Size	Neo4J	AGTS	AeonG	STGraph		
					Total	GS	TS
SmartBench	S	$4.9 \cdot 10^2$	$1.7 \cdot 10^3$	$5.8 \cdot 10^2$	$4.9 \cdot 10^2$	$1.6 \cdot 10^1$	$4.8 \cdot 10^2$
	M	$5.5 \cdot 10^3$	$3.5 \cdot 10^3$	$2.0 \cdot 10^3$	$9.8 \cdot 10^2$	$3.2 \cdot 10^1$	$9.5 \cdot 10^2$
	L	$4.3 \cdot 10^4$	$5.9 \cdot 10^4$	–	$1.4 \cdot 10^4$	$1.4 \cdot 10^2$	$1.4 \cdot 10^4$
SocialNetwork	S	TODO	TODO	TODO	TODO	TODO	TODO
	M	TODO	TODO	TODO	TODO	TODO	TODO
	L	TODO	TODO	TODO	TODO	TODO	TODO

6.2 Comparison

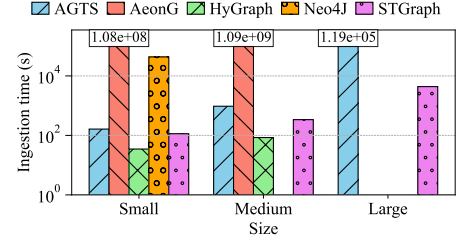
Performance is assessed along three dimensions: ingestion throughput, storage footprint, and query execution time. To ensure a fair comparison, all solutions were evaluated under a single-threaded execution environment and average performance is obtained by aggregating twenty experimental runs. Furthermore, the ingestion process followed a streaming paradigm, where each upsert was executed as an individual transaction whenever computationally feasible. All experiments were conducted using Docker within a single-machine environment (excluding STGraph scale-out tests); consequently, network latency can be considered negligible

6.2.1 Ingestion throughput. Figure 6 reports the ingestion performance. The ingestion of the Medium dataset was omitted for Neo4J due to excessive execution time, the Large dataset was omitted for AeonG due to excessive execution time and for HyGraph due to out-of-memory exception.

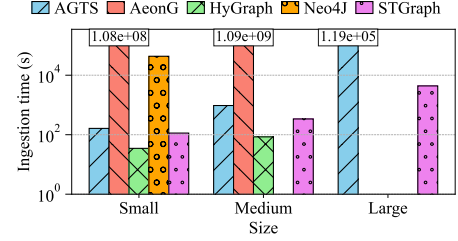
AGTS x mf: forse ha senso graficare, per chi le abbiamo (no AeonG, no HyGraph ma per hygraph volendo posso fare ri-girare a tempo perso) lo stacked bar chart TS/grafico: in AGTS si vede molto bene che per il dataset piccolo caricare il grafo in streaming costa tanto quanto caricare tutte le TS (fare streaming su un B+Tree fa schifo) mentre l’incremento di tempistiche sulle TS è lineare rispetto all’aumentare dell’adimensione del dataset (da noi è sub-lineare) achieves notable ingestion performances

Neo4J graph DBMSs struggle to sustain streaming ingestion workloads and Neo4J achieves the second worst performances when loading the small dataset. For the medium and large datasets, its loading time exceeded the available time and ingestion was performed through static csv imports; results are thus omitted from the comparison.

HyGraph achieves the highest ingestion throughput due to its in-memory design. However, scalability is limited, as ingestion fails with out-of-memory errors on the Large dataset and no ACID properties are guaranteed.



(a) SmartBench



(b) SocialNetwork @mpa: update with new experiments

Figure 6: Ingestion time (s)

AeonG triggers a graph update operation (involving retrieval of the latest version node and management of temporal validity metadata) for each time-series insertion, which introduces additional overhead and significantly longer ingestion times.

STGraph sustains ingestion performance comparable to HyGraph despite relying on disk-based storage and providing ACID guarantees, thanks to its hybrid storage architecture and temporal separation mechanism; while graph updates require versioning operations, time-series ingestion is reduced to lightweight append operations via AsterixDB Data Feeds [27], maintaining high efficiency and a sub-linear increase in ingestion times with respect to dataset sizes.

6.2.2 Storage footprint. Table 6 reports the storage footprint (MB) across dataset sizes (S, M, L), enabling a comparison of compression efficiency and scalability among Neo4J, AGTS, AeonG, and STGraph. HyGraph is omitted due to its in-memory design, which does not provide persistent storage measurements, and AeonG is excluded for the Large dataset as ingestion exceeded the execution limit, indicating scalability limitations.

?? spiegare il risparmi di spazio in termini di edge non salvati/virtuali ??

@mpa integrare quando ci sarà il nuovo dataset

For the Small dataset, Neo4J ($4.9 \cdot 10^2$) and STGraph ($4.9 \cdot 10^2$) achieve the lowest storage footprint, indicating comparable compression efficiency. AeonG ($5.8 \cdot 10^2$) introduces moderate overhead, due to temporal versioning metadata, while AGTS ($1.7 \cdot 10^3$) exhibits the highest footprint, suggesting less compact storage. For the Medium dataset, STGraph ($9.8 \cdot 10^2$) becomes the most space-efficient system, requiring substantially less storage than AeonG ($2.0 \cdot 10^3$), AGTS ($3.5 \cdot 10^3$), and Neo4J ($5.5 \cdot 10^3$). For the Large dataset,

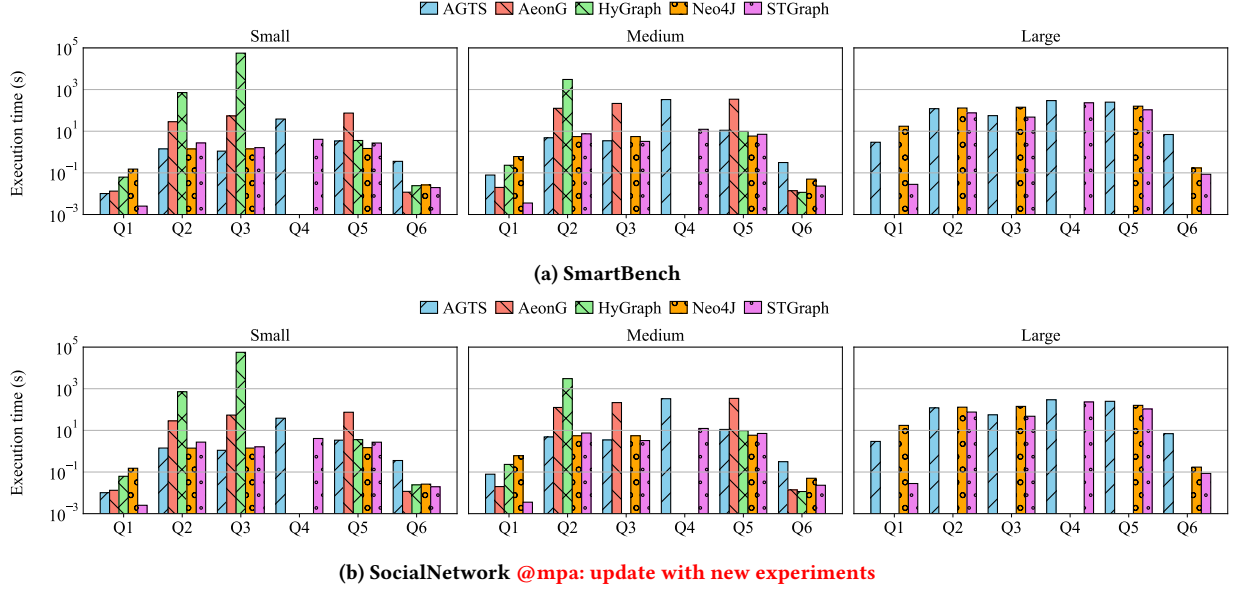


Figure 7: Execution time (s)

STGraph ($1.4 \cdot 10^4$) maintains the smallest footprint by a wide margin compared to Neo4J ($4.3 \cdot 10^4$) and AGTS ($5.9 \cdot 10^4$). While Neo4J and AGTS exhibit substantial growth in storage requirements as the dataset size increases, STGraph shows a more controlled and near-linear growth pattern. This advantage stems from its hybrid storage layout and, in particular, from the fact that STGraph does not materialize explicit graph edges toward individual time-series events. Instead, time-series data are maintained separately and accessed through lightweight references and indexing structures. As the number and length of time series increase, this design prevents the explosion of edge materialization and significantly reduces memory footprint.

These results highlights the effectiveness of its hybrid storage layout and the physical separation between graph and time-series data, which reduces graph-materialization overhead and improves compression. The absence of edge materialization toward time-series events becomes increasingly beneficial at larger scales.

6.2.3 Query execution time. Figure 7 highlights the average execution time.

AGTS outperforms AeonG and Neo4j in analytical queries (Q2, Q3) by leveraging TimescaleDB for time-series management. In these scenarios, the efficiency of the dedicated time-series engine compensates the cost of graph data retrieval. Conversely, performance degrades in graph-intensive workloads (Q1, Q6) where the underlying relational modeling of graph data becomes a bottleneck compared to native graph storage engines.

Neo4J performs best on the small dataset, benefiting from index-free adjacency and an optimized execution engine. However, scalability is limited, particularly for time-series-intensive workloads. While edge traversal remains $O(1)$, the absence of native temporal indexing within adjacency structures

often requires linear scan of historical versions, reducing the effectiveness of pointer-based navigation at scale.

HyGraph Leveraging an in-memory architecture, HyGraph outperforms native graph DBMSs on small-scale topological queries (Q1, Q6). However, it falters on hybrid workloads (Q2, Q3, Q5) due to the computational overhead inherent in its Python implementation. On the medium dataset, several queries exceeded time constraints, while the large dataset proved unfeasible.

AeonG achieves the lowest performance in time-series-oriented queries (Q2, Q3, Q5), reflecting the overhead of its snapshot-based physical design when handling large historical datasets. Nonetheless, it remains competitive in structural graph queries (Q1, Q6), where the number of retrieved historical versions is limited.

STGraph exhibits increasing advantages as dataset volume and time-series cardinality grow. While the small dataset limits the benefits of its multistore architecture (due to query rewriting costs and indexing each time-series separately), performances scale significantly across dataset sizes with respect to both native graph and time series engines, achieving almost a 50% time reduction on the large dataset. Furthermore, STGraph consistently ranks at the top in both graph-only and graph-time-series workloads, showcasing the effectiveness of its hybrid design.

6.3 Scale-out

Scale-out experiments for STGraph were performed on clusters of 2, 4, and 8 nodes, each node featuring the same hardware configuration as the machine used in the comparison tests. The dataset was built on top of the large dataset by increasing the cardinality of each time-series by 5x, for a total of 1.5 billion events spread across 1000

Table 7: Ingestion scale out

Benchmark	Size	Time (s)	#Machines		
			2	4	8
SmartBench	XL	Elapsed Time (s)	$1.0 \cdot 10^4$	$5.7 \cdot 10^3$	$3.6 \cdot 10^3$
		GS Time (s)	$8.4 \cdot 10^1$	$1.1 \cdot 10^2$	$1.3 \cdot 10^2$
		TS Time (s)	$1.0 \cdot 10^4$	$5.6 \cdot 10^3$	$3.4 \cdot 10^3$

Table 8: Query execution scale out (s)

Benchmark	Size	ID	#Machines		
			2	4	8
SmartBench	XL	Q2	$4.6 \cdot 10^1$	$2.8 \cdot 10^1$	$1.8 \cdot 10^1$
		Q3	$9.5 \cdot 10^0$	$1.0 \cdot 10^1$	$1.1 \cdot 10^1$
		Q4	$2.9 \cdot 10^2$	$1.7 \cdot 10^2$	$8.0 \cdot 10^1$
		Q5	$4.8 \cdot 10^1$	$3.0 \cdot 10^1$	$2.0 \cdot 10^1$

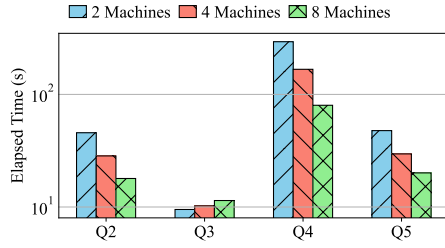


Figure 8: Scale-out query performances varying the number of nodes in AsterixDB cluster. Q1 and Q6 kept out since only graph query.

time-series (see Table 3). Consequently, Q1 and Q6 were excluded from evaluation since they do not entail time-series workload.

Results show a consistent linear scaling with respect to the number of nodes in AsterixDB cluster, with Q2, Q4, Q5 achieving 50% performance boost when scaling from two to four nodes and ???% from four to eight nodes. Q3 exhibits a slight performance degradation as the cluster size increases. This is a consequence of the query being rewritten and delegated to the TS Query Executor as a set of simple existence checks, thereby making it largely independent from time-series cardinality. In this case, the coordination and network shuffling introduced by distributing data across multiple nodes slightly outweighs the benefits of parallelism. This finding indicates that for certain classes of queries, STGraph is able to achieve high efficiency even when dealing with large time-series without the need to rely on horizontal scaling.

7 CONCLUSION AND FUTURE RESEARCH DIRECTIONS

- noi non modelliamo direttamente le trajectory
- non supportiamo continous query.
- Noi abbiamo dei nodi time series che hanno informazioni varie al loro interno. Come in lakehouse, conviene caricare i metadati/statistiche all'interno del nodo del grafo per limitare gli accessi.

REFERENCES

- [1] Sherif Sakr, Angela Bonifati, Hannes Voigt, Alexandru Iosup, Khaled Ammar, Renzo Angles, Walid G. Aref, Marcelo Arenas, Maciej Besta, Peter A. Boncz, Khuzaima Daudjee, Emanuele Della Valle, Stefania Dumbrava, Olaf Hartig, Bernhard Haslhofer, Tim Hegeman, Jan Hidders, Katja Hose, Adriana Iamnitchi, Vasiliki Kalavri, Hugo Kapp, Wim Martens, M. Tamer Özsu, Eric Peukert, Stefan Plantikow, Mohamed Ragab, Matei Ripeanu, Semih Salihoglu, Christian Schulz, Petra Selmer, Juan F. Sequeda, Joshua Shinavier, Gábor Szárnyas, Riccardo Tommasini, Antonino Tumeo, Alexandru Uta, Ana Lucia Varbanescu, Hsiang-Yun Wu, Nikolay Yakovets, Da Yan, and Eiko Yoneki. The future is big graphs: a community view on graph processing systems. *Commun. ACM*, 64(9):62–71, 2021.
- [2] Maciej Besta, Robert Gerstenberger, Emanuel Peter, Marc Fischer, Michal Podstawski, Claude Barthels, Gustavo Alonso, and Torsten Hoeftler. Demystifying graph databases: Analysis and taxonomy of data organization, system designs, and graph queries. *ACM Comput. Surv.*, 56(2):31:1–31:40, 2024.
- [3] Sara Montagna, Angelo Croatti, Alessandro Ricci, Vanni Agnoletti, Vittorio Albarello, and Emiliano Gamberini. Real-time tracking and documentation in trauma management. *Health informatics journal*, 26(1):328–341, 2020.
- [4] Luis Emmi, Roemi Fernández, Pablo Gonzalez-de Santos, Matteo Francia, Matteo Golfarelli, Giuliano Vitali, Hendrik Sandmann, Michael Hustedt, and Merve Wollweber. Exploiting the internet resources for autonomous robots in agriculture. *Agriculture*, 13(5):1005, 2023.
- [5] Yuanyuan Tian. The world of graph databases from an industry perspective. *SIGMOD Rec.*, 51(4):60–67, 2022.
- [6] Angela Bonifati, M. Tamer Özsu, Yuanyuan Tian, Hannes Voigt, Wenyan Yu, and Wenjie Zhang. A roadmap to graph analytics. *SIGMOD Rec.*, 53(4):43–51, 2024.
- [7] Søren Keiser Jensen, Torben Bach Pedersen, and Christian Thomsen. Time series management systems: A survey. *IEEE Trans. Knowl. Data Eng.*, 29(11):2581–2600, 2017.
- [8] Patrick E. O’Neil, Edward Cheng, Dieter Gawlick, and Elizabeth J. O’Neil. The log-structured merge-tree (lsm-tree). *Acta Informatica*, 33(4):351–385, 1996.
- [9] Chen Wang, Xiangdong Huang, Jialin Qiao, Tian Jiang, Lei Rui, Jinrui Zhang, Rong Kang, Julian Feinauer, Kevin Mcgrail, Peng Wang, Diaohan Luo, Jun Yuan, Jianmin Wang, and Jianguang Sun. Apache iotdb: Time-series database for internet of things. *Proc. VLDB Endow.*, 13(12):2901–2904, 2020.
- [10] Sattam Alsubaiee, Yasser Altowim, Hotham Altwaijry, Alexander Behm, Vinayak R. Borkar, Yingyi Bu, Michael J. Carey, Inci Cetindil, Madhusudan Cheelangi, Khurram Faraaz, Eugenia Gabrielova, Raman Grover, Zachary Heilbron, Young-Seok Kim, Chen Li, Guangqiang Li, Ji Mahn Ok, Nicola Onose, Pouria Pirzadeh, Vassilis J. Tsotras, Rares Vernica, Jian Wen, and Till Westmann. Asterixdb: A scalable, open source BDMS. *Proc. VLDB Endow.*, 7(14):1905–1916, 2014.
- [11] Tuomas Pelkonen, Scott Franklin, Paul Cavallaro, Qi Huang, Justin Meza, Justin Teller, and Kaushik Veeraraghavan. Gorilla: A fast, scalable, in-memory time series database. *Proc. VLDB Endow.*, 8(12):1816–1827, 2015.
- [12] Neo4j, Inc. *Neo4j Graph Database Platform*, 2025. Accessed: 2025-09-24.
- [13] Dingfeng Mo, Junfeng Liu, Fan Wang, and Siqiang Luo. Aster: Enhancing lsm-structures for scalable graph database. *Proc. ACM Manag. Data*, 3(1):12:1–12:26, 2025.
- [14] Song Yu, Shufeng Gong, Qian Tao, Sijie Shen, Yanfeng Zhang, Wenyan Yu, Pengxi Liu, Zhixin Zhang, Hongfu Li, Xiaojian Luo, Ge Yu, and Jingren Zhou. Lsmgraph: A high-performance dynamic graph storage system with multi-level CSR. *CoRR*, abs/2411.06392, 2024.
- [15] Ariel Debrouvier, Eliseo Parodi, Matías Perazzo, Valeria Soliani, and Alejandro A. Vaisman. A model and query language for temporal graph databases. *VLDB J.*, 30(5):825–858, 2021.
- [16] Huanhuan Wu, James Cheng, Yiping Ke, Silu Huang, Yuzhen Huang, and Hejun Wu. Efficient algorithms for temporal path computation. *IEEE Trans. Knowl. Data Eng.*, 28(11):2927–2942, 2016.
- [17] Konstantinos Semertzidis and Evaggelia Pitoura. Top-k durable graph pattern queries on temporal graphs. *IEEE Trans. Knowl. Data Eng.*, 31(1):181–194, 2019.
- [18] Youshan Miao, Wentao Han, Kaiwei Li, Ming Wu, Fan Yang, Lidong Zhou, Vijayan Prabhakaran, Enhong Chen, and Wenguang Chen. Immortalgraph: A system for storage and analysis of temporal graphs. *ACM Trans. Storage*, 11(3):14:1–14:34, 2015.
- [19] Maria Massri, Zoltán Miklós, Philippe Raipin Parvédy, and Pierre Meye. Clock-g: Temporal graph management system. *Trans. Large Scale Data Knowl. Centered Syst.*, 54:1–40, 2023.
- [20] Jiamin Hou, Zhanhao Zhao, Zhouyu Wang, Wei Lu, Guodong Jin, Dong Wen, and Xiaoyong Du. Aeong: An efficient built-in temporal support in graph databases. *Proc. VLDB Endow.*, 17(6):1515–1527, 2024.
- [21] Glenn Galvizo and Michael J. Carey. Graphix: "one user's JSON is another user's graph". In *40th IEEE International Conference on Data Engineering, ICDE 2024, Utrecht, The Netherlands, May 13-16, 2024*, pages 3070–3083. IEEE, 2024.

- [22] Mouna Ammar, Shubhangi Agarwal, Angela Bonifati, and Erhard Rahm. Transforming time series into graphs and vice versa with hygraph. In Matthias Boehm and Khuzaima Daudjee, editors, *Proceedings of the Workshops of the EDBT/ICDT 2025 Joint Conference co-located with the EDBT/ICDT 2025 Joint Conference, Barcelona, Spain, March 25, 2025*, volume 3946 of *CEUR Workshop Proceedings*. CEUR-WS.org, 2025.
- [23] Michael Stonebraker and Lawrence A. Rowe. The design of postgres. In Carlo Zaniolo, editor, *Proceedings of the 1986 ACM SIGMOD International Conference on Management of Data, Washington, DC, USA, May 28-30, 1986*, pages 340–355. ACM Press, 1986.
- [24] Nadime Francis, Alastair Green, Paolo Guagliardo, Leonid Libkin, Tobias Linddaaker, Victor Marsault, Stefan Plantikow, Mats Rydberg, Petra Selmer, and Andrés Taylor. Cypher: An evolving query language for property graphs. In *Proceedings of the 2018 international conference on management of data*, pages 1433–1445, 2018.
- [25] Richard Cyganiak. A relational algebra for sparql. *Digital Media Systems Laboratory HP Laboratories Bristol. HPL-2005-170*, 35(9), 2005.
- [26] Sattam Alsubaiee, Alexander Behm, Vinayak R. Borkar, Zachary Heilbron, Young-Seok Kim, Michael J. Carey, Markus Dreseler, and Chen Li. Storage management in asterixdb. *Proc. VLDB Endow.*, 7(10):841–852, 2014.
- [27] Raman Grover and Michael J. Carey. Data ingestion in asterixdb. In Gustavo Alonso, Floris Geerts, Lucian Popa, Pablo Barceló, Jens Teubner, Martín Ugarte, Jan Van den Bussche, and Jan Paredaens, editors, *Proceedings of the 18th International Conference on Extending Database Technology, EDBT 2015, Brussels, Belgium, March 23-27, 2015*, pages 605–616. OpenProceedings.org, 2015.
- [28] Peeyush Gupta, Michael J. Carey, Sharad Mehrotra, and Roberto Yus. Smart-bench: A benchmark for data management in smart spaces. *Proc. VLDB Endow.*, 13(11):1807–1820, 2020.